

Document Production (C.0)

Technical User Guide

The Document Production Technical User Guide defines the job level rules, image level rules, field rules, recipient level rules, and Gen-print Callback rules, included in the CSC value-add layer of Document Production. In addition, this document gives details on the certified mail function, the Sort-Split function, and the Document Production Driver - RUNDAP.

Revision date - April 23, 2008



© 2008 Computer Sciences Corporation. El Segundo, California. All rights reserved. No part of this publication may be reproduced by any means without written permission from Computer Sciences Corporation. Printed in U.S.A. All questions regarding this documentation should be routed through customer assistance, Blythewood, SC, 803-333-6200 or email pcssupport@csc.com.

Table of Contents

Chapter 1: OFFSET Processing.....	7
Overview.....	7
Definition	7
Offset File Definition	8
Single Record Search Criteria: Definition.....	9
Other Syntax	9
Multiple Record Search Criteria.....	11
Search Criteria with Variable Substitution.....	11
Secondary Search Criteria Using Occurrence Value.....	12
Original Numeric Offsets Processing and Multiple Search Criteria	12
 Chapter 2: Job Level 1 Rule Definitions.....	 14
Processing Diagram	15
Job Level Rule Format	16
Job Level 1 Rules (Job).....	17
CreateDDTVarList	19
CSCCCreateDDTVarList.....	19
CSCCInitMailingList	21
CUSAltSearchRec.....	23
CSCCAltSearchRec.....	23
AltSearchRec	23
CUSDumpXml.....	25
CSCCDumpXml	25
CUSInitCertMail	27
CSCCInitCertMail	27
CUSInitFormCache.....	29
CSCCInitFormCache.....	29
LoadEXTOFFS	30
CSCCLoadEXTOFFS.....	30
 Chapter 3: Job Level 2 Rule Definitions.....	 32
Processing Diagram	33
Job Level Rule Format	34
Job Level 2 Rules (Formset)	35
ClearDDTVarList	37
CSCCAddExtrRecs	38
CSCCCheckForImages	48
CSCCCheck4WipForms	49
CSCCCreateMiniExtrFiles.....	50
CSCCCreateMiniExtrFilesWC.....	52

CSCCDumpMiniExtractListsToFiles.....	54
CSCCDumpNewWipXML.....	55
CSCCInsertAGTBanner	57
CSCCMailingListToExtract	58
CSCCReplaceTXTMergeFlds	60
CSCCSummaryTOTRecs.....	61
CSCCUpdateRecipListGVM	62
CUSCertMailToExtract	63
CSCCCertMailToExtract	63
CUSDelForm.....	65
CSCCDelForm	65
CUSDumpXml.....	67
CSCCDumpXml	67
CUSExt2Gvm2	69
CSCCExt2Gvm2.....	69
CUSSetFieldData.....	73
CSCCSetFieldData	73
CSCCSetPageNumFieldData.....	78
CSCCSetNonPageNumFieldData	79
CSCCSetPageNumForGroup.....	80
CUSImportDPXMLFile	82
CSCCImportDPXMLFile.....	82
CUSSendToWIP.....	84
CSCCSendToWIP	84
CUSTbl2Gvm	85
CSCCTbl2Gvm.....	85
Chapter 4: Job Level 3 Rule Definitions.....	88
Processing Diagram	89
Job Level Rule Format	90
Job Level 3 Rules (Image)	90
CSCCConvertImgVar	92
ConvertImgVar.....	92
ConvertImageVar.....	92
Chapter 5: Job Level 4 Rule Definitions.....	94
Processing Diagram	95
Job Level Rule Format	96
Job Level 4 Rules (Field).....	97
ConvertVar	98
Chapter 6: Image Level Rule Definitions.....	100
Processing Diagram	101
Image Level Rule Format.....	102
Image Rule Format Conventions.....	102
Image Level Rules	103
CSCCConvertImgVar_Run.....	105
ConvertImgVar_Run	105

CSCCCreateSubExtractList.....	107
CSCCEnumerateGraphics.....	109
CSCCInlineBitmap.....	111
CSCCSetExtrFile.....	113
CUSNYBarcode	115
IMVAR	117
UpdateOvRecsUsed	118

Chapter 7: Field Rule Definitions 120

Processing Diagram	121
Field Rule DDT Line Definition.....	121
Field Rule Format.....	123
Field Rule Format Conventions	124
Field Level Rules.....	126
DecrOvRecsUsed.....	128
GetRecSum.....	130
totalem.....	130
movedate.....	133
CSCCmovedate	133
numcomp.....	139
tbllook	141
VAR.....	145
CSCCSetExtractFL	149

Chapter 8: Recipient Level Rule Definitions 151

Recipient Level Rules	151
CmpOvFlw	153
CSCCCmpOvFlw.....	153
CUSmultirec	156
CSCCmultirec	156
CUSLoadAttachedForms	158
CSCCLoadAttachedForms	158
CSCCTableCntr.....	161
PNTTableCntr.....	161
TableCntr	161

Chapter 9: Genprint Callback Rule Definitions..... 162

Processing Flow	163
Callback Format	164
Callback Functions	165
CreateBalance.....	167
CreateLineWithGVM.....	168
CreateMailCounter.....	169
CreatePageCounter	170
CUSDrawOMRMark	171
CUSFilterForms	172
CUSImageCreateHeader	173
CUSImagePrintName	174

CUSMultiCallbacks.....	175
CUSMultiFilePrint.....	176
CUSPlaceTotalPage	177
PrintPolicyJacket	178
SortFormsForRecip	179
Chapter 10: Hook Procedure Rule Definitions.....	180
Overview.....	180
Implementation.....	180
Hook Functions	182
Chapter 11: Certified Mail Implementation.....	188
User Workflow	189
Technical Overview.....	191
Input.....	192
Errors and Error Handling	192
Output.....	192
Implementation Instructions.....	192
FSISYS.INI Settings	192
CERT_FIELDS Group	192
CERTMAIL Group	193
CertRecip:<Recipient> Group	194
TRNDFDFL.DFD Entries	195
AFGJOB.JDT Entries.....	196
<Base Rules>	196
<Base Form Set Rules>.....	197
Chapter 12: Sort/Split Functions	199
FSISYS.INI.....	200
CSCCTrnfilePopulate Hook Procedure	200
CSCCSortTrnFile Hook Procedure	200
CSCCSortBatchFiles Hook Procedure.....	201
Chapter 13: Document Production Driver - RUNDAP.....	206
Features.....	206
Inventory.....	207
Execution.....	207
Implementation.....	208
Set-Up to Run All Document Production Modules on the PC	208
Set-Up to Run WIP/Archive Modules Only on the PC	215
Archive/Upgrades Distribution	217
Messages.....	220
Distribution Log Messages.....	220
RunDAP Error Messages.....	221
RunDAP Error Dialog Messages.....	225
History of Enhancements	229
Release 1.1 Modifications.....	229
Release 1.2 Modifications.....	230

Release 1.3 Modifications.....	230
Release 1.4 Modifications.....	230
Release 1.5 Modifications.....	231
Release 1.6 Modifications.....	231
Release 1.7 Modifications.....	231
Release 1.8 Modifications.....	231
Release 1.9 Modifications.....	233
Release 1.10 Modifications	233

Chapter 14: Agent Email Notification and Interface to Media Management Inbox 235

Overview.....	235
Installation and Configuration.....	235
Configuring AGTEMAIL.INI	236
Troubleshooting.....	244

Chapter 15: Maintenance Utilities..... 245

CheckEJP	245
Overview.....	245
Usage	245
CheckOVF.....	246
Overview.....	246
Usage	246
FindCAR.....	247
Overview.....	247
Usage	247
NoUserEDITUTIL.....	247
Overview.....	247
Usage	248

Chapter 16: Document Change Log 250

Appendix A: ARCRET Utility 251

Chapter 1: OFFSET Processing

Overview

The original Document Production system only required a numeric offset value for its data retrieval processing. This was the offset into the record where the data resided. You can still use numeric offsets as before. However, if the data on the record was moved or other data was inserted before this data, then the numeric offset values would have to be changed to reflect the correct location of the data. This must be done everywhere the original offset value was used. This can result in maintenance issues.

CSC has developed a methodology to externalize and isolate the offset values for all of the possible records in the extract file. This is what we refer to as OFFSET processing. This methodology connects a specific piece of extract file data to a variable field with minimal or no DDT mapping maintenance. The system administrator can make a change to the offsets file and have that change effect all DDT fields that use that offset without having to edit any DDT file.

Primary offset files are provided as part of Document Production's base system. The primary file contains the majority of offsets and is produced through utility programs according to the host policy processing system. Secondary files consist of all hand-coded offsets and offsets for records created during GENDATA execution.

Definition

The offset file contains a list of field names (column names) and their offset value into the record for all records that contain information needed by Document Production. Each rule that needs information from a policy record calls a generic function called GENGetRecord, which uses the information in the data portion on the DDT file to retrieve the appropriate record from the internal pol-

icy record list. The data area is composed of Search Criteria and additional information specific to the rule. The Search Criteria is defined as a comma-delimited list of Offset Tag Name and Key Values used to retrieve the proper record from the internal policy list. To understand how GENGetRecord interprets the information coded in the data portion of a DDT, FDT, or JDT rule, see the following DDT entry.

During Document Production processing, data is accessed on a specific record using an offset value. The Offset Tag Names and Values defined for all policy data files are contained in the offset file. This file is supplied with the base system. Additional offset files can be loaded. The base system provides a manually created offset file, PGMOFFF.DAT. See AFGJOB for offset file examples. If database definitions change, only these tables need to be updated for Document Production.

Offset File Definition

Excerpt from the POINT IN offset file PTDOFFFL.dat:

```

      1          2          3
1234567890123456789012345678901234567890. . .etc.
00017 00010 PMSP0000TBLOFF
00027 00003 PMSP0000SYMBOL
00030 00007 PMSP0000POLICY0NUM
00037 00002 PMSP0000MODULE

```

This data is generated by utilities according to the host system. Manually created offsets can also be loaded.

Position	Description
1 - 5	Offset value
6	Space
7 - 11	Length
12	Space
13 - 72	Table Name and Column Name (no embedded spaces)

Single Record Search Criteria: Definition

The following bullets describe the processing steps on this example:

```
;0;1;FLA A;0;2;FLD;0;2;;move_it;  
TBLOFF,PMSP1200,USE0CODE,(MT,SM,TM);N;N;N;N;
```

- GENGetRecord uses the offset file to translate the field name (column name) into an offset value.
- Each policy record is examined in order to see if that key value (PMSP1200) exists at the specific OFFSET, represented by TBLOFF, into the record.
- If all search conditions are found TRUE on a specific record, then that record is returned for use by the rule.
- Specifically, if there is a record in the internal policy list that has "PMSP1200" at the OFFSET value represented by TBLOFF and has "MT," "SM," or "TM" at the offset value represented by PMSP1200,USE0CODE, then that record is returned to the rule.

The first offset key value is retained as the record name. All subsequent field names use that record name and field name to translate the offset value in the offset file.

Other Syntax

Example 1

```
;TBLOFF,PMSP1200,USE0CODE,~(MT,SM,TM);
```

Searches for a PMSP1200 record with something other than MT, SM or TM at the offset represented by USE0CODE.

Example 2

There are four coverages:

- EQ
- EQA
- EQB
- EQC

To return the record with the coverage EQ, code the data portion as follows:

```
;TBLOFF,ASBYCPL1,BYAOTX,EQ,BYAOTX,~(EQA,EQB,EQC);
```

Searches for an ASBYCPL1 record where BYAOTX = 'EQ' and not 'EQA' and not 'EQB' and not 'EQC'.

If BYAOTX,~(EQA,EQB,EQC) is omitted, then the returned record could contain any of the four coverages because they all begin with EQ.

Example 3

```
;TBLOFF,PMSP0000,TYPE0ACT,EN,ERROR014,~SPACES;
```

Searches for a PMSP0000 record where TYPE0ACT equals EN and the entire ERROR014 field is not equal to SPACES. It can be used with or without the "~" character.

Example 4

```
;TBLOFF,PMSP0000,TYPE0ACT,EN,ERROR014,~___;
```

Note: There are three underscore characters in this example. Underscores can be used with or without the "~" character. Parenthesis can also be used, for example ~(___).

Searches for a PMSP0000 record where TYPE0ACT equals EN and the first three positions of the ERROR014 field is not equal to SPACES. The underscore represents a space and is translated back to a space during the record search.

Example 5

```
;TBLOFF,ASBECPL1,BEB8NB,FORM_A;
```

Note: There is one underscore character in FORM_A.

Searches for an ASBECPL1 record where BEB8NB equals "FORM A." The underscore represents a space and is translated back to a space during the record search.

Multiple Record Search Criteria

Sometimes it is necessary to check for multiple conditions on multiple records before a rule can complete its processing. Each component of the Search Criteria must be coded in the data area. An "|" separates each component. The following is an example of a multiple record search:

```
;TBLOFF,ASBBCPL1,BBAGX,CA|TBLOFF,ASB5CPL1,B5ANTX,CA;
```

- The first component is interpreted as the Primary Search Criteria. If all other Search Criteria are met, the record requested by the Primary Search Criteria is returned to the rule.
- Every component after the first is assumed to have the same SrcRecNum as the Primary component unless otherwise specified.

Search Criteria with Variable Substitution

```
TBLOFF,ASBBCPL1,BBAGX,CA|TBLOFF,ASB5CPL1,B5ANTX,CA,  
B5BRNB,00000, B5AENB,{Unit},B5I5TX,Y;
```

```
TBLOFF,ASBBCPL1,BBAGX,CA|TBLOFF,ASB5CPL1,B5ANTX,CA,  
B5BRNB,00000, B5AENB,<<Unit>>,B5I5TX,Y;
```

- The first component is interpreted as the Primary Search Criteria. If all other Search Criteria are met, the record requested by the Primary Search Criteria is returned to the rule.
- Every component after the first is assumed to have the same SrcRecNum as the Primary component unless otherwise specified.
- Indicate the Variable Substitution with "<<>>." The legacy characters "{}" are also supported.

Secondary Search Criteria Using Occurrence Value

```
TBLOFF, ASBBCPL1, BBAGTX, CA| 5, TBLOFF, ASB5CPL1, B5ANTX, CA,
    B5BRNB, 0000, B5AENB, {Unit}, B5I5TX, Y;
```

```
TBLOFF, ASBBCPL1, BBAGTX, CA| 5, TBLOFF, ASB5CPL1, B5ANTX, CA,
    B5BRNB, 0000, B5AENB, <<Unit>>, B5I5TX, Y;
```

- The first component is interpreted as the Primary Search Criteria. If all other Search Criteria are met, the record requested by the Primary Search Criteria is returned to the rule.
- Every component after the first is assumed to have the same SrcRecNum as the Primary component unless otherwise specified.
- Indicate the Variable Substitution with "<<>>". The legacy characters "{}" are also supported.

Original Numeric Offsets Processing and Multiple Search Criteria

The original all numeric offset processing does not support multiple search criteria processing as described in [Search Criteria with Variable Substitution](#) or in [Secondary Search Criteria Using Occurrence Value](#).

For example, if writing the last example using only numeric offsets, it would look like this:

```
17, ASBBCPL1, 43, CA| 5, 17, ASB5CPL1, 56, CA, 46, 0000, 62,
    {Unit}, 346, Y;
```

If there was a specific occurrence of the secondary record sought, then this example would work properly. However this scenario is the exception rather than the rule. This example would more likely be coded without the occurrence value:

```
17, ASBBCPL1, 43, CA| 17, ASB5CPL1, 56, CA, 46, 0000, 62,
    {Unit}, 346, Y;
```

The base system processing would interpret the offset value of 17 as an occurrence value rather than an offset and report a coding or syntax error.

Possible Solutions

- 1** Disable the Job Level 1 rule CUSAltSearchRec in AFGJOB. Your current all numeric offsets will work, but you will lose the multiple record search functionality.
- 2** Enable the Job Level 1 rule CUSAltSearchRec in AFGJOB, but have a minimal number of offsets to satisfy the record occurrence and offset value in all secondary search criteria. For example:

```
17,ASBBCPL1,43,CA| 5,TBLOFF,ASB5CPL1,56,CA,46,0000,62,  
  {Unit},346,Y| 5,TBLOFF,...etc;  
17,ASBBCPL1,43,CA| TBLOFF,ASB5CPL1,56,CA,46,0000,62,  
  {Unit},346,Y| TBLOFF,...etc;
```

This will satisfy the syntax requirements of the CUSAltSearchRec rule and provide proper results.

Chapter 2: Job Level 1 Rule Definitions

The DEFLIB member, AFGJOB.jdt, contains the list of job level rules that are executed during GENDATA.EXE. The four levels of rules are processed in a particular order. The <Sections> are as follows:

<Base Rules>	This base (this implementation) uses these rules.
<Base Form Set Rules>	Every form set in this base uses these rules.
<Base Image Rules>	Every image in this base uses these rules.
<Base Field Rules>	Every field in this base uses these rules.

This chapter is concerned with the rules under the <Base Rules> section header. These rules are executed in the order they appear, processing pre/post B messages according to the diagram below.

The placement of the rule under a particular <Section> in AFGJOB and the messages it receives determine what type of processing occurs, not the level indicator in the rule. Each level receives a different set of messages.

You may include comments in AFGJOB.JDT. Comment lines must be on separate lines and in the following format:

```
/* This is a valid comment line      */
/*;CreateGlbVar;1;TXTLst,PVOID;
;CreateGlbVar;1;TXTLst,PVOID; /* This is an invalid comment line */
```

Processing Diagram

Job Level 1 rules are contained in, but not limited to, rulbas.c, cusjdt.c, csccejdt.c, pntjdt.c, or clntjdt.c in the order shown below. They receive message B. Base Job Rules, level 1, are called once at the start of the job and once at the end of the job.

Base Rules (JL 1) - PRE B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - PRE A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Image (JL 3) - PRE A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Image - PRE B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - PRE C	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - PRE A	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Field - PRE B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - PRE C	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - POST C	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Field - POST B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - POST A	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Image (JL 3) - POST C	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Image - POST B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - POST A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - POST A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Rules (JL 1) - POST B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt

Job Level Rule Format

Rule Name

Job Level 1 Rule Name Tag

Usage

Generic/Specific

Program

program.c

Function Name

Function name

Level

1

Message

RP_PRE_PROC_B

RP_POST_PROC_B

Processing description and pertinent information.

Parameter	Max Length	Usage
Rule Name	32	Rule name
Level Number	1	Job rule level number
Add'l Data	1024	Additional data, if necessary

Examples

```
;CreateGlbVar;1;TXTLst,PVOID;
;Ext2Gvm;2;TBLOFF,PX002001 LOB,3,LOB;
;RULStandardImageProc;3;Always the first image level rule;
;RULStandardFieldProc;4;Always the first field level rule;
```

Further Explanation of Examples, if necessary.

Job Level 1 Rules (Job)

The following table describes all the Job Level 1 rules and where each rule resides. If the function name is different from the rule name, it is noted in the description. For more discussion of the base rules, single transaction processing, 2-UP printing, and OMR Marks refer to Skywire documentation.

Messages: RP_PRE_PROC_B
RP_POST_PROC_B

This section addresses only those rules that CSC added to CUSLIB and CSCCLIB. It defines each function and explains how it is used in the system.

BASE Rule Name	Function	PGM	Description
CreateDDTVarList CSCCCreateDDTVarList	CSCCProcessDDTVarList	csccfld	Creates DDT VAR list.
CSCCInitMailingList	CSCCInitMailingList	csccmal	Initializes the Multi-Mail List subsystem and automatically creates the MML offset file. The offset file will contain the record offsets for the MML mail-type record(s) and the CERTSTAT record.
CUSAltSearchRec CSCCAltSearchRec AltSearchRec	CSCCAltSearchRec	cscgetrc	Installs the offset method of retrieving extract records.
CUSDumpXml CSCCDumpXml	CSCCDumpXml	cscxml	Writes data to an XML file in preparation for Agent Email.
CUSInitCertMail CSCCInitCertMail	CSCCInitCertMail	csccert	Set up and terminate Certified Mail Processing.
CUSInitFormCache CSCCInitFormCache	CSCCSInitFormCache	cscjdt	Set up and destroy attached form cache.

BASE Rule Name	Function	PGM	Description
LoadEXTOFFS CSCCLoadEXTOFFS	CSCCLoadEXTOFFS	cscclfd	Loads user defined offsets file(s) into VMMList.

CreateDDTVarList

CSCCCreateDDTVarList

Rule Name

CreateDDTVarList
CSCCCreateDDTVarList

Usage

Generic

Program

cscfld.c

Function Name

CSCCProcessDDTVarList

Message

RP_PRE_PROC_B
RP_POST_PROC_B

Processing Description

CreateDDTVarList creates and stores DDTVarList using the global variable, DDTVarList. This rule must be executed before the image-level rule *IMVAR* or the field-level rule *VAR* can be used.

You must create the global variable called DDTVarList first for this rule to work. See the following statement.

```
;CreateGlbVar;1;DDTVarList,PVOID; must precede this rule.
```

Preprocessing

Create and make available DDTVarList for *IMVAR* and *VAR* processing.

Postprocessing

Destroy list and free memory resources.

Parameter	Usage
Rule Name	CreateDDTVarList
Level	1
Add'l Data	Not Used

Example

```
;CreatedDDTVarList;1;;
```

CSCCInitMailingList

Rule Name

CSCCInitMailingList
InitMailingList

Usage

Generic

Program

csccmail.c

Function Name

CSCCInitMailingList

Message

RP_PRE_PROC_B
RP_POST_PROC_B

Preprocessing

This Job Level 1 rule initializes the Multi-Mail List sub system and automatically creates the MML offset file. The offset file will contain the record offsets for the MML mail-type record(s) and the CERTSTAT record.

It adds the MML transaction record as the last record in the Trn-File.dat. This transaction record causes the MML to be processed.

Postprocessing

Destroys the list(s) created during this process.

Notes

IF THIS RULE IS OMITTED THE MULTI-MAIL LIST (MML) MODULE WILL NOT WORK. All of the other MML job-rules will be skipped (if not excluded from the AfgJob.jdt file).

The MML TrnFile transaction has a Y in the MailList field. This is necessary in order to trigger the processing of the MML information that was gathered during the policy processing.

The TrnFile fields that represent the TransactionId and Transaction-Type fields ([DocSetNames] options) will be set to "MAILLIST" and "CM" respectively on default if different values are not provided (see Control Groups and Options below).

When the MML TrnFile transaction is processed, the MML extract-records list gathered (during policy processing) is processed as a regular transaction.

CSCCInitMailingList must appear before the "JobInit1" base job level rule so that it can update the TrnFile.dat before JobInit1 opens the file exclusively, because after a file is open exclusively it cannot be updated.

The offset filename passed as a parameter should not be one of the existing offset filenames (like PTDOFFFL.DAT), because this rule will initialize the offset file specified before it adds the MML records offset entries.

Parameter	Usage
Rule Name	CSCCInitMailingList
Level	1
Add'l Data	Offset File for Multi-Mail List processing

Example

```
;CSCCInitMailingList;1;CERTOFFS.DAT;
```

CUSAltSearchRec

CSCCAltSearchRec

AltSearchRec

Rule Name

CUSAltSearchRec
CSCCAltSearchRec
AltSearchRec

Usage

Generic

Program

cscgetrc.c

Function Name

CSCCAltSearchRec

Message

RP_PRE_PROC_B

Processing Description

This Job Level 1 rule controls several base processes.

- Replaces the Skywire base functions, located in GENLIB, for retrieving records from the extract file and installs the offset file getrecord processing, located in CUSLIB.
- Intercepts and provides preprocessing for several RULLIB rules, which were previously modified as part of the POINT IN Value Add layer.

RULLIB rules that are intercepted for preprocessing:

Field Level Rules

concat
DateDiff
if
setaddr
setaddr2
setaddr3

Job Level 2 Rules

Ext2Gvm

GetCo
GetLOB

Recipient Level Rules

recipif

Parameter	Usage
Rule Name	CUSAltSearchRec
Level	1
Add'l Data	Not Used

Example

```
;CUSAltSearchRec;1;;
```

CUSDumpXml

CSCCDumpXml

Rule Name

CUSDumpXml
CSCCDumpXml

Usage

Generic

Program

cscxml.c

Function Name

CSCCDumpXml

Level

1 and 2

Message

RP_PRE_PROC_B (level 1)
RP_POST_PROC_A (level 2)
RP_POST_PROC_B (level 1)

Processing Description

In preparation for running the Agent Email Program, this rule writes data into an XML file. This process is INI driven. The Agent Email program will read the XML file created through this rule rather than accessing the Archive Database files as we have done in previous versions. This makes the Agent Email program independent of the Archive implemented.

JOB Level 1 - PRE Process

Performs the initializations and saves off the information that is in FSISYS.ini file. This way, the INI is read only once.

JOB Level 2 - POST Process

Gets the data from the Global Variables and places it into a keyed list. It also adds the variable GVM.

JOB Level 1 - POST Process

The list is read and the data is written to the XML file. A break on Agent number is used.

Parameter	Usage
Rule Name	CUSDumpXML or CSCCDumpXml
Level	1
Add'l Data	Path and filename for DAP.XML The filename, DAP.xml, is required. DAP.xml resides in the DATA folder.

Parameter	Usage
Rule Name	CUSDumpXml or CSCCDumpXml
Level	2
Add'l Data	Not Used

Example

```
;CUSDumpXml;1;. \DATA\DAP.XML;  
;CUSDumpXml;2;;
```

CUSInitCertMail**CSCCInitCertMail*****Rule Name***

CUSInitCertMail
CSCCInitCertMail

Usage

Generic

Program

cscccert.c

Function Name

CSCCInitCertMail

Message

RP_PRE_PROC_B
RP_POST_PROC_B

Processing Description

This Job Level 1 rule initializes the CertMail subsystem. It creates the CertMail offset file in preparation for the CSCCCertMailToExtract Job Level 2 rule, which creates the CertMail extract records for processing.

Preprocessing

Create CertMail processing structure and initialize with information provided through FSISYS.ini.

Postprocessing

Destroy CertMail processing VMMLists and free memory resources.

Parameter	Usage
Rule Name	CUSInitCertMail or CSCCInitCertMail
Level	1
Add'l Data	<Cert-Offset-file> - The CertMail offset filename to create. File resides in the DEFLIB directory. < NONE > - No CertMail offset file is created. The assumption is that one already exists.

Example

```
;CUSInitCertMail;1;CERTOFFS.DAT;
;JobInit1;1;;
```

Important: This base rule must appear before the JobInit1 base rule in AFGJOBjdt because CUSInitCertMail needs to modify the TRNFILE. JobInit1 opens TRNFILE in an exclusive mode and, therefore, no changes can be made to TRNFILE after JobInit1 executes.

Error Conditions

Message: The TRNDFDFL.DFD CertMail field is missing.

Cause: This must exist in order to initialize the CertMail system.

```
TRNDFDFL.DFD
< FIELDS >
.
.
    FIELDNAME = RunDate
; 2001.2.022
    FIELDNAME = CertMail
.
.
; 2001.2.022
< FIELD:CertMail >
    INT_TYPE = CHAR_ARRAY
    INT_LENGTH = 2
    EXT_TYPE = CHAR_ARRAY_NO_NULL_TERM
    EXT_LENGTH = 1
    KEY = N
    REQUIRED = N
```

CUSInitFormCache

CSCCInitFormCache

Rule Name

CUSInitFormCache
CSCCInitFormCache

Usage

Generic

Program

cscjdt.c

Function Name

CSCCInitFormCache

Message

RP_PRE_PROC_B
RP_POST_PROC_B

Processing Description

CSCCInitFormCache creates and destroys the cache for attached form lists used with the custom recipient rule, CSCCLoadAttachedForms. This rule must be executed immediately after BuildMasterFormList in AFGJOB.jdt.

Preprocessing

Create cache.

Postprocessing

Destroy cache and free memory resources.

Parameter	Usage
Rule Name	CUSInitFormCache or CSCCInitFormCache
Level	1
Add'l Data	100 – maximum number of lists to cache. If left blank, negative or > 100, value defaulted to 5.

Example

```
;CUSInitFormCache;1;100;
```

LoadEXTOFFS

CSCCLoadEXTOFFS

Rule Name

LoadEXTOFFS
CSCCLoadEXTOFFS

Usage

Generic

Program

cscfld.c

Function Name

CSCCLoadEXTOFFS

Message

RP_PRE_PROC_B
RP_POST_PROC_B

Processing Description

LoadEXTOFFS creates and loads external offsets file(s). Multiple offset files can be loaded. Typical usage would be to load the BASE offsets file first, then any other implementation specific files next. All files must reside in DEFLIB. The entries in the secondary offset file(s) append to or update the information already in the offset hash table.

Offset file may contain comments. Comment lines begin with an asterisk (*).

Preprocessing

Create, load, or update the external offsets hash table with implementation specific offsets file(s).

Postprocessing

Destroy list and free resources.

Parameter	Usage
Rule Name	LoadEXTOFFS
Level	1
Add'l Data	[FileName.Ext]

Example

```
/* Load hash table with primary base Offsets File */
;LoadEXTOFFS;1;PTOFFFFL.DAT;
/* Load hash table with secondary impl/cust specific Offsets File */
;LoadEXTOFFS;1;PGMOFFFFL.DAT;
```

External record format

```
1      2      3
00017 00010 PMSP0200TBLOFF
```

1. offset [5];
Separated by space.
2. length [5];
Separated by space.
3. record/column [60];

OFFSFILE format. (defined in genlib.h) Internal Hash Table entry format

```
typedef struct OFFSTRUCT_TAG {
    int offset;
    int len;
    char Key [60];
} OFFSTRUCT;
```

Chapter 3: Job Level 2 Rule Definitions

The deflib member AFGJOB.JDT contains the list of job-level rules that are executed during GENDAW32.EXE. The four levels of rules are processed in a particular order. The <Sections> are as follows:

<Base Rules>	This base (this implementation) uses these rules.
<Base Form Set Rules>	Every form set in this base uses these rules.
<Base Image Rules>	Every image in this base uses these rules.
<Base Field Rules>	Every field in this base uses these rules.

This chapter is concerned with the rules under the <Base Form Set Rules> section header. These rules are executed in the order they appear, processing pre/post A messages according to the diagram below.

The placement of the rule under a particular <Section> in AFGJOB and the messages it receives determine what type of processing occurs, not the level indicator in the rule. Each level receives a different set of messages.

You may include comments in AFGJOB.JDT. Comment lines must be on separate lines and in the following format:

```
/* This is a valid comment line */
/* ;CreateGlbVar;1;TXTLst,PVOID;
;CreateGlbVar;1;TXTLst,PVOID; /* This is an invalid comment line */
```

Processing Diagram

Job Level 2 rules are contained in, but not limited to, rulbsfl.c, cusjdt.c, csccejdt.c, pntjdt.c, or clntjdt.c, as shown below. They receive message A. Base Formset Rules, level 2, are called before and after each policy transaction is processed.

Base Rules (JL 1) - PRE B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - PRE A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Image (JL 3) - PRE A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Image - PRE B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - PRE C	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - PRE A	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Local Field - PRE B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - PRE C	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - POST	Rulbsfl/Cusjdt/Csccejdt/Pntjdt /Clntjdt
Local Field - POST B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - POST A	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Image (JL 3) - POST C	Rulbsim/CusjdtCsccejdt//Pntjdt/Clntjdt
Local Image - POST B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - POST A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - POST A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Rules (JL 1) - POST B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt

Job Level Rule Format

Rule Name

Job Level 2 Rule Name Tag

Usage

Generic/Specific

Program

Program

Function Name

Function name

Level

2

Message

RP_PRE_PROC_A
RP_POST_PROC_A

Processing description and pertinent information

Parameter	Max Length	Usage
Rule Name	32	Rule name
Level Number	1	Job rule level number
Add'l Data	1024	Additional data, if necessary

Examples

```
;CreateGlbVar;1;TXTLst,PVOID;  
;Ext2Gvm;2;TBLOFF,PX002001 LOB,3,LOB;  
;RULStandardImageProc;3;Always the first image level rule;  
;RULStandardFieldProc;4;Always the first field level rule;
```

Processing explanation for the rule, other notes, tables, disclaimers follow.

Job Level 2 Rules (Formset)

The following table describes all the job level 2 rules and where each rule resides. If the function name is different than the rule name, it is noted in the description. Refer to Skywire documentation for further discussion of the base rules, single transaction processing, 2-UP printing, and OMR marks.

Messages

RP_PRE_PROC_A

RP_POST_PROC_A

This section addresses only those rules that CSC added to CUSLIB and CSCCLIB. It defines each function and explains how it is used in the system.

Base Rule Name	Function	PGM	Description
ClearDDTVarList	CSCCProcessDDTVarList	cscclfd	Clears the contents of DDTVarList for each policy transaction.
CUSCertMailToExtract CSCCCertMailToExtract	CSCCCertMailToExtract	cscclcert	Gathers cert mail data from policies and stores it in the CertMail structure.
CUSDelForm CSCCDelForm	CSCCDelForm	csccljdt	During postprocessing, this rule deletes a specific form from the current formset. This rule is used in conjunction with the ISO form triggering process.
CUSDumpXml CSCCDumpXml	CSCCDumpXml	cscclxml	Writes data to an XML file in preparation for Agent Email.
CUSExt2Gvm2 CSCCExt2Gvm2	CSCCExt2Gvm2	csccljdt	Places data in a global variable.
CUSSendToWIP CSCCSendToWIP	CSCCSendToWIP	csccljdt	Sends a transaction to WIP based on a search criteria.
CUSSetFieldData CSCCSetFieldData	CSCCSetFieldData	csccljdt	Set specific occurrences of a specified field name to the text string passed in to the rule.
CSCCSetPageNumField-Data	CSCCSetPageNumField-Data	csccljdt	For Page Numbering ONLY. This function calculates and prints page numbers by forms. A new page numbering sequence begins for each form.
CSCCSetNonPageNum-FieldData	CSCCSetNonPageNum-FieldData	csccljdt	For All Other NON Page Numbering functions only. This function performs its requested operation on a form-by-form basis.
CSCCSetPageNumForGroup	CSCCSetPageNumForGroup	csccljdt	Page Numbering for a list of "FORMS"

Base Rule Name	Function	PGM	Description
CUSTbl2Gvm CSCCTbl2Gvm	CSCCTbl2Gvm	csccejdt	Places data from a table into a global variable.
CUSImportDPXmlFile CSCCImportDPXmlFile	CUSImportDPXmlFile	csccexml	Imports the XML file created by Document Print into GENDATA.
CSCCAddExtrRecs AddExtrRecs	CSCCAddExtrRecs	cscceaddx	Creates new extract records
CSCCCheckForImages	CSCCCheckForImages	csccejdt	This rule is to be used in conjunction with the Print Transaction Balancing processing. It counts the number of images for the transaction and places the total in a GVM variable.
CSCCCheck4WipForms	CSCCCheck4WipForms	Csccejdt	Determines if the current transaction contains WIP forms. If so, the document is returned to the user in HTM for completion.
CSCCCreateMiniExtrFiles	CSCCCreateMiniExtrFiles	csccemini	Creates and manages mini extract file lists.
CSCCCreateMiniExtr-FilesWC	CSCCCreateMiniExtr-FilesWC	csccemini	Creates and manages mini extract file lists for Work Comp.
CSCCCreateSubExtractList	CSCCCreateSubExtractList	csccefdt	Creates, maintains, and destroys a subset of the extract file and makes it available to GENDATA processing.
CSCCDumpMiniEx-tractListsToFiles	CSCCDumpMiniEx-tractListsToFiles	csccemini	Dumps to file any or all mini extract files created in CSCCCreateMiniExtrFiles rule. Debug tool.
CSCCDumpNewWipXML	CSCCDumpNewWipXML	csccexml	Writes data to an XML file for all newly inserted transactions into the WIP database. This file is included in the Agent Email process.
CSCCInsertAgtBanner	CSCCInsertAgtBanner	csccejdt	Inserts a banner page for Agent Bulk Mailings
CSCCMailingListToExtract	CSCCMailingListToExtract	cscceemail	Gathers and stores Mail List data in preparation for the Mail List Report processing after all transactions are processed.
CSCCReplaceTXTMergeFlds	CSCCReplaceTXTMergeFlds	csccejdt	This rule is only used with Document Print. It prepares images with the "can grow" attribute to accommodate the data entered.
CSCCSummaryTOTRecs	CSCCSummaryTOTRecs	csccemini	Summarizes CPP coverage data and creates new record for extract file.
CSCCUpdateRecipListGVM	CSCCUpdateRecipListGVM	csccejdt	This rule populates a GVM with a list of recipients for each transaction, which is stored in the WIP and ARCHIVE index databases.

ClearDDTVarList

Rule Name

ClearDDTVarList

Usage

Generic

Program

cscfild.c

Function Name

CSCCProcessDDTVarList

Level

2

Message

RP_POST_PROC_A

Processing Description

This rule clears DDTVarList variables between policies.

Preprocessing

Not applicable

Postprocessing

Clear the entire DDT VAR List by setting it to NULL.

Parameter	Usage
Rule Name	ClearDDTVarList
Level	2
Add'l Data	Not Used

Example

```
;ClearDDTVarList;2;;
```

CSCCAddExtrRecs

Rule Name

CSCCAddExtrRecs

Usage

Generic

Program

CscAddX

Function Name

CSCCAddExtrRecs

Level

2

Message

RP_PRE_PROC_A

Processing Description

With this feature, you can create new extract records, during the GENDATA process, based off of existing extract records.

The [AddExtrRecs] group allows you to add new records to the policy extract list handle. These records are derived from existing policy extract records.

With this feature you can:

- Split a single record (like ASBQCPL1) into multiple records, which can be used for overflow. You can create overflow occurrences for a single record.
- Build unique records from existing records.
- Create a new record with the necessary data, based on conditions of different records or the relationship between two records. This is like joining two different records and selecting the necessary data fields to build the new record.

Note: The symbolic offset field names and their offset values for the newly created record need to be added manually to an offset file.

To activate this feature:

AFGJOB.JDT

Add the base formset job rule "AddExtrRecs" after the LoadExtractData job rule and before the LoadRcpTbl job rule in AfgJob.jdt.

;AddExtrRecs;2;;

INI File

Add [AddExtrRecs] group and options.

Preprocessing

Create new extract records, during the GENDATA process, based off of existing extract records.

Postprocessing

Not applicable

Parameter	Usage
Rule Name	CSCCAddExtrRecs
Level	2
Add'l Data	Not Used

There are three types of [AddExtrRecs] formats:

- Record Definition Entry:

Defines the new record name and the main condition under which the new record will be added. If this condition fails, the field-definition entries will not be processed. You should have only one record-definition entry for each new record.

- Field-Definition Entry:

Defines the source record (through a source record condition) to extract the data from the destination record. Only when the source record condition is through is the new record created with the data specified.

- The source data field(s) to extract the data from or the string constant to move to the destination field(s).
- You may have multiple field definition entries for each new record.

- Unique-Field-Definition Entry:

Defines the source record and the source key-fields that make the record unique. This entry should be the only entry/option for a record definition entry if the objective is to create unique records from existing records.

Record Definition Entry Format

Syntax:

```
EXTRREC = ;<LOB-List>;<Rec-Name>; [<Trig-Cond>|UNIQUE];
```

Note: The data/value for the option begins and ends with a semi-colon. A semi-colon delimits each parameter in the option data/value.

ENTRY OPTION

EXTRREC	(Required) Keyword that identifies the record-definition entry. Only one entry for each new record.
---------	---

ENTRY VALUE / PARAMETERS

LOB-List	(Optional) Comma delimited list of LOBs. The new records will be appended to the extract list of those LOBs listed (for example, APV,HP.) If left blank, the new record could be generated for all LOBs.
Rec-Name	(Required) Name of the user-defined record to create destination record.
Trig-Cond Or UNIQUE	(Required) A True/False condition to determine when to create the destination record. This is the main-condition. Reserved Keyword - when specified will build unique records form existing records using the specified unique fields. See the "Unique Field Definition Entry Format."

Field-Definition Entry Format

Syntax:

```
<Rec-Name> = ;<LOB-List>;<SearchMask>;(<DestFld>=<DestData>[:])...;
```

Note: The data/value for the option begins and ends with a semi-colon. A semi-colon delimits each parameter in the option data/value. To define more than one DestFld, separate each with two colons "::".

ENTRY OPTION

Rec-Name	(Required) Name of user-defined record specified in the Record Definition Entry above.
----------	--

ENTRY VALUE / PARAMETERS

LOB-List	(Optional) Comma delimited list of LOBs. The new records will be appended to the extract list of those LOBs listed (for example, APV,HP.) If left blank, the new record could be generated for all LOBs.
SearchMask	(Required) Search condition to find the source record to extract the source data from for the destination field and/or foreign-key. (Destination field data can also come from a record other than the source record.) If you want to search for only a single occurrence of a source record, then specify the occurrence number. Examples: TBLOFF,ASBACPL1 ' get all occurrences 3,TBLOFF,ASBACPL1 ' only get 3rd occurrence See Parameter Syntax on page 43 for detail description.
DestFld	(Required) Field name on the Rec-Name record. Note: The offsets for the Rec-Name fields should be added to the offset file manually.

DestData	(Required) Value to copy to the DestFld. There are three types of formats that can be used to supply the data: • String constant enclosed in quotes (e.g.: "Y") • Source-field data (for example, USE0CODE,2 or 42,2) • Secondary source record (may use foreign-key to retrieve secondary record) See Parameter Syntax on page 43 for detail description.
----------	---

Unique-Field-Definition Entry Format

Note: Only one entry must exist for each unique record entry.

Syntax:

```
Rec-Name = ;<LOB-List>;<SearchMask>;keyfld1[::][keyfld2::]...;
```

Note: The line begins and ends with a semi-colon. A semi-colon delimits each parameter. Within the key field parameter, if there are more than one key-field option, then separate them with two colons "::".

ENTRY OPTION

Rec-Name	(Required) Name of user-defined record specified in the Record Definition Entry above.
----------	--

ENTRY VALUE / PARAMETERS

LOB-List	(Optional) Comma delimited list of LOBs. The new records will be appended to the extract list of those LOBs listed (for example, APV,HP.) If left blank, the new record could be generated for all LOBs.
SearchMask	(Required) Search condition used to select source records to be used in the unique record process. There are no destination fields. Once it has been established that the source record is unique, the entire source record is copied to the new record. The offset file entries for the source record can be duplicated and the record name changed to the new record-name to create the offset entries for the new record-name. See Parameter Syntax on page 43 for detail description.

keyfld1 - keyfldn	(At least one key1fld Required) SrcFld,SrcLen Source field names (or offset and length) that identify the uniqueness of the source record. Keyfields are separated by two consecutive colons, "::".
-------------------	--

Parameter Syntax

SearchMask syntax:

```
[rec-occ,]{tblOff|TBLOFF},tblName[, {fldOff|fldName},fldVal]...
```

rec-occ	(Optional) a positive integer used to identify the record occurrence. Do not use in a Unique-Field Definition Entry.
tblOff or TBLOFF	(Required) A positive integer representing the record-name offset. (Reserved Keyword) A Symbolic reference for the record-name offset value in the offset file.
tblName	(Required) Name of the source record to search for.
fldOff or fldName	(Optional) A positive integer representing Source-field offset. A symbolic reference for the source-field offset value in the offset file.
fldVal	(Optional) Data/value to search for at the fldOff or fldName offset

DestData Syntax:

```
{"<string>"| SrcFld,SrcLen[, ]...|<IF-Condition>:trueValue:falseValue}
```

"string"	(Required) Use this format to copy a string constant to the DestFld
or SrcFld,SrcLen	Use this format to copy a source-field, or concatenate several source-fields, into the DestFld.
SrcFld	Symbolic or numeric offset value of the source field.
SrcLen	Number of characters to copy from the source-record to DestFld starting at SrcFld offset.

Or	
<IF-Condition>	Note: The IF-Condition must be within angle brackets "<.>".
:trueValue:falseValue	Use this format to copy data from a secondary source-record, which could be derived from a relationship between the secondary and primary source records through a simple or complex foreign keys. A string could be copied instead of data from the secondary record if the relationship condition fails or succeeds. If the secondary source-record in the relationship condition is the same as the primary source-record, then no foreign key is necessary.

<IF-Condition> Syntax:

```
<[rec-occur, ] {tbl2Off|TBLOFF}, tbl2Name[, {fld2Off|fld2Name}, /
{fld2Value|foreign-key}]...>
```

Note: The IF-Condition must be within angle brackets "<.>".

rec-occ	(Optional) a positive integer used to identify the record occurrence.
tbl2Off or TBLOFF	(Required) A positive integer representing the Secondary record-name offset. (Reserved Keyword) A Symbolic reference for the secondary record name offset value in the offset file.
tbl2Name	(Required) Secondary source-name to search for.
fld2Off or fld2Name	(Required) A positive integer representing secondary source-field offset. A symbolic reference for the secondary source-field offset value in the offset file.

fld2Value or foreign-key	(Optional) Value to search for at fld2Off or fld2Name offset. The primary source-field that is related to the secondary source field. They contain the same type of data that allows for the relationship. The current primary source-field offset and number of characters to use in retrieving the related secondary source-record.
--------------------------------	--

foreign-key Syntax:

{SrcFld, SrcLen}

Note: Foreign-key must be in curly braces in order to morph the key into the actual data.

trueValue/falseValue Syntax:

{"<string>" | [1|2,] (SrcFld, SrcLen[,]) ... }

"string"	A string constant in quotes (for example, "Valued Customer") to be copied to the DestFld.
SrcFld	Source-field symbolic name or numeric offset.
1,SrcFld	If SrcFld is prefixed with 1 (1,SrcFld), then the SrcFld must be a field or offset from the primary source-record.
2,SrcFld	If SrcFld is prefixed with 2 (2,SrcFld), then the SrcFld must be a field or offset from the associated secondary source-record.
SrcLen	Number of characters to extract from source-record starting at SrcFld offset.

Example 1

```
EXTRREC = ;HP;ADDLINFO;TBLOFF,PMSP0000,TYPE0ACT,~BI;
ADDLINFO = ;HP;TBLOFF,ASBACPL1,BABTVA,~000000000000;
          CRDESC="Auto/Home Discount":CRPREM=BABTVA,11;
```

Example 1 explanations

In this example, the ADDLINFO records will be added to the extract list handle when the transaction type is not BI for a Homeowner policy.

Destination fields CRDESC, CRPREM, and PREMINC are fields on the ADDLINFO record.

For each occurrence of TBLOFF,ASBACPL1 where BABTVA is not 00000000000, an ADDLINFO record will be added to the extract list handle with:

"Auto/Home Discount" in the CRDESC field, and 11 characters from the ASBACPL1 record, starting at the BABTVA offset, will be copied to the CRPREM field.

Example 2

```
EXTRREC = ;APV;POLDRV;TBLOFF,ASBLCPL1|TBLOFF,PMSP0000,TYPE0ACT,~BI;
POLDRV  = ;APV;TBLOFF,ASBLCPL1;DRVID=BLA0NB,5: :\
POLDRV  = DRIVER=<TBLOFF,ASBMCPL1,BMBKTX,(EXH,MIN,URN),BMA0NB,{BLA0NB,5}>: \
POLDRV  = "N":BLC7ST,1;
```

Example 2 explanations

In this example, the POLDRV records will be added to the extract list handle when the transaction type is not BI for an APV policy.

Destination fields DRVID and DRIVER are fields on the POLDRV record.

For each occurrence of the ASBLCPL1 record, a POLDRV record will be added to the extract list handle.

Five characters from the current ASBLCPL1 record, starting at the BLA0NB offset, will be copied to DRVID field.

DRIVER field will be set to "N" if the condition between brackets "<>" is true. If the condition is false, one character from the current ASBLCPL1 record, starting at the BLC7ST offset, will be copied to the DRIVER field.

The condition between brackets "<>" uses {BLA0NB,5} as a foreign key to find the corresponding ASBMCPL1 for the current ASBLCPL1. This foreign key establishes a relationship between the ASBMCPL1 record and the ASBLCPL1 record through the driver-id field in both records (BMA0NB and BLA0NB, respectively).

Also note the use of the of the continuation character '\'. The line ends with backslash '\'.

The next line continues after the equal sign. "POLDRV =" is required.

Example 3

```
EXTRREC = ;APV;PMSP12UN;UNIQUE;
```

```
PMSP12UN = ;APV;TBLOFF,PMSP1200,USE0CODE,(AI,LP);\
```

PMSP12UN =
USE0CODE,2::DESC0LINE1,30::DESC0LINE2,30::DESC0LINE3,30::\

PMSP12UN = DESC0LINE4,30::DZIP0CODE,10;

Example 3 explanations

Unique PMSP1200 records are created based on the additional interest code, name, and address. This could eliminate the generation of multiple copies of the policy for the same additional interest just because they are listed as the additional interest on more than one vehicle.

USE0CODE, DESC0LINE1, DESC0LINE2, DESC0LINE3, DESC0LINE4, and DZIP0CODE are the key fields.

Note: The backslash means continuation. This is a way of spreading a long line on multiple lines.

CSCCCheckForImages

Rule Name

CSCCCheckForImages

Usage

Generic

Program

cscjdt.c

Function Name

CSCCCheckForImages

Message

RP_POST_PROC_A

Processing Description

This rule is to be used in conjunction with the Print Transaction Balancing processing. It counts the number of images for the transaction and places the total in a GVM variable. The Update_PRTBAL1 DAL script uses this information to populate the POL_HAD_NO_IMGS_IND field in the PRTBAL1 database.

Parameter	Usage
Rule Name	CSCCCheckForImages
Level	2
Add'l Data	Not Used

Example

```
<Base Form Set Rules>
;PostTransDAL;2;CALL("Update_PRTBAL1");
;CSCCCheckForImages;2;;
;RULStandardTransactionProc;2;;
```

CSCCCheck4WipForms

Rule Name

CSCCCheck4WipForms

Usage

Specific

Program

cscjdt.c

Function Name

CSCCCheck4WipForms

Message

RP_POST_PROC_A

Processing Description

This Rule is used for Document Print documents. It determines if the current transaction contains WIP forms by checking the manual batch flag. If the manual flag is set to "Y," then the PRTTYPE is changed to HTM so the document can be produced in HTML format.

This rule was written for the POINT IN application for their immediate print process through Document Print. This process creates a DEC in PDF format, but if the DEC contains WIP forms, the document should be created in HTML so the user can complete the document.

Note: This rule is only applicable to Immediate Print and therefore is placed in AFGDEC.jdt. The placement of the rule is important and must be placed after the PaginateAndPropagate rule.

Parameter	Usage
Rule Name	CSCCCheck4WipForms
Level	2
Add'l Data	Not Used

Example

```
;CSCCCheck4WipForms;2;;
```

CSCCCreateMiniExtrFiles

Rule Name

CSCCCreateMiniExtrFiles

Usage

Generic

Program

Cscmini.c

Function Name

CSCCCreateMiniExtrFiles

Message

RP_PRE_PROC_A

RP_POST_PROC_A

Processing Description

This Job Level 2 Rule creates, stores, clears, and ultimately destroys the new "mini" extract file lists. AFGJOB creates some of the list names managed in this rule. This rule will populate and clear these lists, but the ultimate destruction of the list handle will occur during normal Job Level 1 processing.

Preprocessing

Creates and stores the new "mini" extract file lists.

Postprocessing

Clears all "mini" list and destroys the list handles and GVMs that were created in preprocessing. Some lists are only cleared and remain for the next transaction. These list handles are destroyed during normal Job Level 1 postprocessing. Restores the original extract list handle for the next image.

Parameter	Usage
Rule Name	CSCCCreateMiniExtrFiles
Level	2
Add'l Data	Not Used

Example

```
;CSCCCreateMiniExtrFiles;2;;
```

CSCCCreateMiniExtrFilesWC

Rule Name

CSCCCreateMiniExtrFilesWC

Usage

Generic

Program

Cscmini.c

Function Name

CSCCCreateMiniExtrFilesWC

Message

RP_PRE_PROC_A

RP_POST_PROC_A

Processing Description

Applies only to Work Comp.

This Job Level 2 Rule creates, stores, clears and ultimately destroys the new "mini" extract file lists. AFGJOB creates some of the list names managed in this rule. This rule will populate and clear these lists, but the ultimate destruction of the list handle will occur during normal Job Level 1 processing.

Preprocessing

Creates and stores the new "mini" extract file lists.

Postprocessing

Clears all "mini" list and destroys the list handles and GVMs that were created in preprocessing. Some lists are only cleared and remain for the next transaction. These list handles are destroyed during normal Job Level 1 postprocessing. Restores the original extract list handle for the next image.

Parameter	Usage
Rule Name	CSCCCreateMiniExtrFilesWC
Level	2
Add'l Data	Not Used

Example

```
;CSCCCreateMiniExtrFilesWC;2;;
```

CSCCDumpMiniExtractListsToFiles

Rule Name

CSCCDumpMiniExtractListsToFiles

Usage

Generic

Program

Cscmini.c

Function Name

CSCCDumpMiniExtractListsToFiles

Message

RP_PRE_PROC_A

Processing Description

This Job Level 2 Rule dumps the contents of the “Mini” extract lists to individual files. The name of the list is the name of the file. You can dump ALL of the lists or a specific list.

Parameter	Usage
Rule Name	CSCCDumpMiniExtractListsToFile
Level	2
Add'l Data	Name of List or ALL

Example

```
;CSCCDumpMiniExtractListsToFiles;2;Common;  
;CSCCDumpMiniExtractListsToFiles;2;ALL;
```

CSCCDumpNewWipXML

Rule Name

CSCCDumpNewWipXML

Usage

Generic

Program

cscxml.c

Function Name

CSCCDumpNewWipXML

Level

1 and 2

Message

RP_PRE_PROC_B (Level 1)

RP_POST_PROC_B (Level 1)

RP_POST_PROC_A (Level 2)

Postprocessing

In preparation for running the Agent Email Program, this rule dumps data for each transaction sent to WIP into an XML file (NEW-WIP.XML). This process is INI file driven. The Agent Email program uses the file created from this rule as an input file. The transactions contained in this file will be sent to the user whose email address is associated with the New Wip transaction.

JOB Level 1 – PRE Process

Performs the initializations and saves off the information that is in the FSISYS.ini file.

JOB Level 2 – POST Process

Gets the data from the GVMs and places it into a keyed list.

JOB Level 11 – POST Process

The list is written to the XML file. A break on User ID is used.

Parameter	Usage
Rule Name	CSCCDumpNewWipXML
Level	1

Parameter	Usage
Add'l Data	filename – NEWWIP.xml

Parameter	Usage
Rule Name	CSCCDumpNewWipXML
Level	2
Add'l Data	Not Used

Example

```
;CSCCDumpNewWipXML;1;NEWWIP.XML;
```

Note: *NEWWIP.XML is the default filename and is also defined in AGTEMAIL.INI. If you change this filename, please change it in both places (AFGJOB.jdt and AGTEMAIL.ini)*

```
;CSCCDumpNewWipXML;2;;
```

CSCCInsertAGTBanner

Rule Name

CSCCInsertAGTBanner

Usage

Generic

Program

cscjdt.c

Function Name

CSCCInsertAGTBanner

Level

2

Message

RP_PRE_PROC_A

RP_POST_PROC_A

Processing Description

CSCCInsertAGTBanner inserts an agent mailer page in front of the first agent formset in the agent batch. This functionality assumes that the TRNFILE is sorted according to Agent Number. The process is <product>sys.ini file controlled. See POINTSYS.ini file, <InsertAGTBanner> group for example.

Parameter	Usage
Rule Name	CSCCInsertAGTBanner
Level	2
Add'l Data	Not Used

Example

```
;CSCCInsertAGTBanner;2;;
```

Note: This rule usually appears after the UpdatePOLFile rule in AFG-JOB. If the INI file group does not exist, then this function is skipped.

CSCCMailingListToExtract

Rule Name

CSCCMailingListToExtract
MailingListToExtract

Usage

Generic

Program

csccmail.c

Function Name

CSCCMailingListToExtract

Message

RP_PRE_PROC_A
RP_POST_PROC_A

Preprocessing

After the last policy is processed the accumulated data is written out as extract records for processing on the pre process phase.

The CSCCInitMailingList rule wrote a MailList record to TrnFile. This record serves as the end of process indicator for this rule.

The PRE_PROC section of this rule is executed only when the MailList record is detected. The POST_PROC section is executed on all records except the MailList TrnFile record.

Note

THIS RULE MUST APPEAR AFTER LoadExtractData RULE and before any rule that sets the GroupName1 and GroupName2 values in the pRPS structure for proper operation.

Postprocessing

This base form set rule gathers defined mailing list data from the policies being process and stores it into the MailList VMM Handle during the post process phase.

Note

This rule must appear somewhere after the LoadExtractData rule and before any batch assignment rule (like AssignBatWithTbl, or AssignToBatch and/or IfRecipUsed) because the policy extract records have to be loaded prior to the transaction being processed.

CSCCMailingListToExtract must appear before any batch assignment rule because the PRE PROCESS part of **CSCCMailingListToExtract** needs to switch the extract-list first so that the MML transaction is assigned to the correct batch.

Parameter	Usage
Rule Name	CSCCMailingListToExtract
Level	2
Add'l Data	Not Used

Example

```
;CSCCMailingListToExtract;2;;
```

CSCCReplaceTXTMergeFlds

Rule Name

CSCCReplaceTXTMergeFlds

Usage

Generic

Program

cscjdt.c

Function Name

CSCCReplaceTXTMergeFlds

Message

RP_POST_PROC_A

Processing Description

This rule is only used with Document Print.

It resizes images that have the "can grow" attribute to their maximum size. Entered data is then merged with the image (See CSCCImportDPXMLFile). The images are then resized and the coordinates adjusted to fit properly on the page. It is easier to shrink an image then to increase its size after the formset has already been built.

Parameter	Usage
Rule Name	CSCCReplaceTXTMergeFlds
Level	3
Add'l Data	Not Used

Example

```
<Base Image Rules>
;RULStandardImageProc;;Always the first image level rule;
;CSCCReplaceTXTMergeFlds;3;;
```

CSCCSummaryTOTRecs

Rule Name

CSCCSummaryTOTRecs

Usage

Generic

Program

Cscmini.c

Function Name

CSCCSummaryTOTRecs

Message

RP_PRE_PROC_A

Processing Description

This Job Level 2 Rule reads through the extract file once and summarizes coverage premiums by State/InsLine/Coverage and writes out a record to the extract file with that data.

Only applicable to BOP and CPP Line of Business.

Parameter	Usage
Rule Name	CSCCSummaryTOTRecs
Level	2
Add'l Data	Not Used

Example

```
;CSCCSummaryTOTRecs;2;;
```

CSCCUpdateRecipListGVM

Rule Name

CSCCUpdateRecipListGVM

Usage

Generic

Program

cscjdt.c

Function Name

CSCCUpdateRecipListGVM

Message

RP_POST_PROC_A

Processing Description

This rule populates a GVM with a list of recipients for each transaction, which is stored in the WIP and ARCHIVE index databases. The purpose of this rule is to increase performance of Media Management's retrieval of transactions from WIP and Archive by eliminating the need to read the associated CARFILE for archive and *.POL file for WIP.

Implementing this rule will require an ARCHIVE Index (APPIDX) and WIP Index (WIP) migration. Refer to the **Document Production Migration Guide** for complete instructions.

Parameter	Usage
Rule Name	CSCCUpdateRecipListGVM
Level	2
Add'l Data	Name of the GVM to hold the data. This GVM name must be defined in the appropriate DFD file.

Example

```
;CSCCUpdateRecipListGVM;2;RecipList;
```

CUSCertMailToExtract**CSCCCertMailToExtract*****Rule Name***

CUSCertMailToExtract
CSCCCertMailToExtract

Usage

Generic

Program

cscccert.c

Function Name

CSCCCertMailToExtract

Level

2

Message

RP_POST_PROC_A

Processing Description

CSCCInitCertMail writes a CERTMAIL record to the TrnFile. This record serves as the end-of-process indicator for this rule. When the CERTMAIL record is detected, only then is the PRE_PROC section of this rule executed. The POST_PROC section is executed on all records except the CERTMAIL record.

Preprocessing

After the last policy is processed the accumulated data is written out as extract records for processing on the pre process phase.

Postprocessing

Gather cert mail data from policies being process and store it into the CertMail structure. The POST_PROC section is executed on all records except the CERTMAIL record.

Parameter	Usage
Rule Name	CUSCertMailToExtract or CSCCCertMailToExtract
Level	2

Parameter	Usage
Add'l Data	Not Used

Example

```
;CUSCertMailToExtract;2;;
```

Important: This rule must appear after the *LoadExtractData* rule and before any batch assignment rules for proper operation.

Error Conditions

If CertMail is not initialized, this rule fails.

CUSDelForm

CSCCDelForm

Rule Name

CUSDelForm
CSCCDelForm

Usage

Generic

Program

cscjdt.c

Function Name

CSCCDelForm

Level

2

Message

RP_POST_PROC_A

Processing Description***Preprocessing***

Not applicable

Postprocessing

During postprocessing this rule deletes a specific form from the current formset.

Parameter	Usage
Rule Name	CUSDelForm or CSCCDelForm
Level	2
Add'l Data	FormName (Third parameter in FORM.DAT)

Example

```
;CUSDelForm;2;BANNER;  
;CUSDelForm;2;HODEC PAGE;
```

FORM.DAT

```
;BASE;HOMEOWNERS;BANNER;BANNER;R;;BANNERHD|DOX<INS(1)>;  
;BASE;HOMEOWNERS;HODEC PAGE;Homeowners Dec;RD;;  
COMMONHD|DOX<INS(1),AGT(1),MT(0),SM(0),TM(0),LP(0),AI(0),COM(1)>\  
/HODEC|DS<INS(1),AGT(1),MT(0),SM(0),TM(0),LP(0),AI(0),COM(1)>\  
/HODEC2|DS<INS(1),AGT(1),MT(0),SM(0),TM(0),LP(0),AI(0),COM(1)>\  
/COMMONFT|DSOY<INS(1),AGT(1),MT(0),SM(0),TM(0),LP(0),AI(0),COM(1)>;
```

CUSDumpXml

CSCCDumpXml

Rule Name

CUSDumpXml
CSCCDumpXml

Usage

Generic

Program

cscxml.c

Function Name

CSCCDumpXml

Level

1 and 2

Message

RP_PRE_PROC_B (level 1)
RP_POST_PROC_A (level 2)
RP_POST_PROC_B (level 1)

In preparation for running the Agent Email Program, this rule dumps data into an XML file. This process is INI driven. The Agent Email program will read the XML file created through this rule rather than accessing the Archive Database files. This makes the Agent Email program independent of the Archive implemented.

JOB Level 1 - PRE Process

Performs the initializations and saves off the information that is in FSISYS.ini file. This way, the INI is read only one time.

JOB Level 2 - POST Process

Gets the data from the GVMs and places it into a keyed list. It also adds the variable GVM.

JOB Level 1 - POST Process

The list is read and the data is written to the XML file. A break on Agent number is used.

Parameter	Usage
Rule Name	CUSDumpXml or CSCCDumpXml
Level	1
Add'l Data	Path and filename for DAP.xml. The filename, DAP.xml, is required. DAP.xml resides in the DATA folder.

Parameter	Usage
Rule Name	CUSDumpXml or CSCCDumpXml
Level	2
Add'l Data	Not Used

Example

```
;CUSDumpXml;1;. \DATA\DAP.xml;  
;CUSDumpXml;2;;
```

CUSExt2Gvm2

CSCCExt2Gvm2

Rule Name

CUSExt2Gvm2
CSCCExt2Gvm2

Usage

Generic

Program

cscjdt.c

Function Name

CSCCExt2Gvm2

Level

2

Message

RP_PRE_PROC_A

Processing Description

This rule takes data from the extract list and places it into previously defined global variables. If the data is to have significance during processing, the rule must execute during preprocessing.

Error Conditions

Rule sends an error message and returns msgFAIL under the following conditions:

- No parameters exist or the parameters are incorrectly formatted within the rule area.
- Unable to successfully call GENGetGlbDataPtr()

Note: The rule will not error if record is not found. Returns msgSUCCESS.

Parameter	Usage
Rule Name	CUSExt2Gvm2 or CSCCExt2Gvm2

Parameter	Usage
Level	2
Add'l Data	offset1,string offset2,len(,...),GBLVAR DEFAULT/FMTMASK

Parameter	Description	Req/Opt
offset1	Identifies position of the record to compare Required	
String	String that can be found at offset1	Required
Blank	A space must separate the previous string & Required next string	
offset2	Identifies the start position of the data to be Required copied can be found	
Len	Length of the data to be copied	Required
(,...,)	Can specify more than 1 set of offset,len pair. Optional The data is concatenated together.	
GBLVAR	Global variable that holds the data. Global variable must be created with the "CreateGlb- Var" rule or through a DFD entry.	Required
Blank	A space must separate the global variable and Required default value, if present	
Default	Default value to copy to the global variable if Optional record is not found.	
/	Must be present when specifying format mask.	Required
FMTMASK	Format mask. Individual formats must be sep- Optional arated by a ",". N - Don't trim trailing spaces from data. U - Upper case data.	

Examples

In all examples below, it is assumed that the global variable has already been created with CreateGlbVar rule.

Example 1

Locates extract record matching the search mask TBLOFF,TRAN-REC and adds the value found at the offset represented by OFFSET for length of 25 to the global variable POLNBR. Trailing spaces are trimmed from data.

```
;CUSExt2Gvm2;2;TBLOFF,TRANREC OFFSET,25,POLNBR;
```

Example 2

Concatenates the values found at position 60 for length of 3 and position 71 for length of 25 to the global variable POLSYMNB. Trailing spaces are trimmed before data is concatenated.

```
;CUSExt2Gvm2;2;TBLOFF,TRANREC 60,3,71,25,POLSYMNB
```

Example 3

Same as above but trailing spaces are not trimmed before data are concatenated. The data is also converted to upper case.

```
;CUSExt2Gvm2;2;TBLOFF,TRANREC 60,3,71,25,POLSYMNB /N,U
```

Example 4

Add data found at offset 60 for a length of 2 to global variable BILLIND. Trailing spaces are not trimmed from data. If search mask TBLOFF,TRANREC is not found, default value "99" is used.

```
;CUSExt2Gvm2;2;TBLOFF,TRANREC 60,2,BILLIND 99/N
```

Example 5

The example below shows you how to take data from different records and concatenate them into one global variable. The first line takes data from record TRANREC at offset 60 for a length of 3 and adds it to global variable POLSYMNBMOD. Trailing spaces are not trimmed and the data is converted to upper case.

The second line takes data from record POLICY at offset 52 for length of 25 and concatenates it to the value already in global variable POLSYMNBMOD. Trailing spaces are trimmed.

The last line takes data from record FOLDER at offset 102 for length of 2 and concatenates it to the value already in global variable POLSYMNBMOD. Trailing spaces are trimmed.

If a record is not found, then default value 00 is concatenated to the global variable.

```
;CUSExt2Gvm2;2;TBLOFF,TRANREC 60,3,POLSYMNBMOD /N,U
;CUSExt2Gvm2;2;TBLOFF,POLICY 52,25,POLSYMNBMOD
;CUSExt2Gvm2;2;TBLOFF,FOLDER 102,2,POLSYMNBMOD 00
```

Example 6

The example below shows you how to substitute a literal text when a search condition is TRUE. The text **NOTIR** is added to the GVM when the search data is found. This is just the opposite

of the regular default where the default is used if the data is not found. The regular default is not available when the literal format mask is used.

```
;CUSExt2Gvm2;2;TBLOFF,NRNCANRS,55,~IR 70,3,Feeder4 NOTIR/L;
```

Example 7

```
;CSCCExt2Gvm2;2;TBLOFF,RECA FIELDA,5,TempGVM;  
;CSCCExt2Gvm2;2;TBLOFF,RECB,FIELDB,{TempGVM} FIELDC,10,GVMName;
```

Step 1:

Locates extract record matching the search mask TBLOFF,RECA and places the data at offset FIELDA for a length of five into a Global Variable called TempGVM.

Step 2:

Data contained in TempGVM is retrieved and replaces "{TempGVM}" in the search mask. Then an extract record matching the "Morphed" search mask is located and places the data at offset FIELDC for a length of 10 into a Global Variable called GVMName.

CUSSetFieldData

CSCCSetFieldData

Rule Name

CUSSetFieldData

CSCCSetFieldData

Program

cscjdt.c

Usage

Generic

Function Name

CSCCSetFieldData

Level

2

Message

RP_PRE_PROC_A

Set specific occurrences of a specified field name to the text string passed to the rule.

Preprocessing**Postprocessing**

Not applicable

Parameter	Usage
Rule Name	CUSSetFieldData
Level	2
Add'l Data	[Form]\[Image]\[Field]\[Text]\[When]\[Flags]\

Example

```
;CUSSetFieldData;2;
FIELD->PAGENUMBER\TEXT->%d of %d\FLAGS->PAGE_NUMBERS_USING_TOTAL\;
;CUSSetFieldData;2;;FORM->MYFORM\IMAGE->MYIMAGE\FIELD->CONTINUED\
TEXT->(continued)\WHEN->POST_PROC\FLAGS->ALL_OCCURRENCES\;
```

The above example sets the text of all occurrences of the field called CONTINUED on the image(s) called MYIMAGE on the form(s) called MYFORM to the string (continued).

FORM and IMAGE are optional. If FORM is not specified, all forms are assumed. Likewise, if IMAGE is not specified, all images are assumed.

Valid when strings are PRE_PROC and POST_PROC, which determine at what time during the rules processor run the field will get set.

Valid FLAGS strings are:

ALL_MATCHES

ONLY_LAST_MATCH

ONLY_FIRST_MATCH

PAGE_NUMBERS

PAGE_NUMBERS_USING_TOTAL

PAGE_NUMBERS_USING_TOTAL_REVERSE

The above flags cannot be used with each other. Only one may be specified per call to the rule.

The first three flags must be self-explanatory. They determine what fields (of those found matching the FIELD name specified) get set to the string pointed to by TEXT.

The PAGE_NUMBERS and PAGE_NUMBERS_USING_TOTAL flags are used when you want to fill the fields with the current page number. In these cases, the string pointed to by TEXT needs to contain special characters to denote where the page number must go and where the optional number of total pages must go. These special characters must be in the following format:

```
%??(flags??)??(width??)d
```

The percent sign (%) lets the rule know that you want it to insert a number in that position (either the page number or the total number of pages, depending on the FLAG you passed in).

An optional character ??(flags??) can come next. They can be any one of the following (notice that more than one flag can display):

Flag	Meaning	Default
-	Left Justify the result within the field width.	Right Justify.
+	Prefix the output value with a sign (+ or -).	Sign displays only for negative signed values (-).
blank	Prefix the output value with a blank if the output value is signed and positive. The+ flag overrides the blank flag if both display, and a positive signed value is output with a sign.	No blank.
0	The 0 flag (zero) causes leading 0's to pad the output Space padding. to the field width. The 0 flag is ignored if precision is specified for an integer or if the “-“ flag is specified.	

Next, you can specify an optional (width) parameter.

(Width) is a nonnegative decimal integer controlling the minimum number of characters printed. If the number of characters in the output value is less than the specified width, blanks are added on the left or the right (depending on whether the “-” flag is specified) until the minimum width is reached.

(Width) never causes a value to be truncated; if the number of characters in the output value is greater than the specified width, or width is not given, all characters of the value are printed (subject to the precision specification).

The letter d must be used to tell the rule you are finished printing the number. It can then continue looking at the following data as a regular text string until it gets to the next percent (%) sign.

Because the percent (%) symbol is being used as a special symbol, if you want to actually print a % symbol for any reason, you can just specify two percent symbols back-to-back.

Examples of 'TEXT' parameter when using one of the PAGE_NUMBERS options:

%d (with FLAGS specified as PAGE_NUMBERS)

2

3

%d of %d (with FLAGS specified as
PAGE_NUMBERS_USING_TOTAL)

1 of 3

2 of 3

3 of 3

%d/%d (with FLAGS specified as PAGE_NUMBERS_USING_TOTAL)

1/3

2/3

3/3

%02d of %02d (with FLAGS as PAGE_NUMBERS_USING_TOTAL)

01 of 03

02 of 03

03 of 03

%2d,%2d (with FLAGS as
PAGE_NUMBERS_USING_TOTAL_REVERSE)

3, 1

3, 2

3, 3

When to use PAGE_NUMBERS, PAGE_NUMBERS_USING_TOTAL, or
PAGE_NUMBERS_USING_TOTAL_REVERSE:

If you want both the current page number as well as the total number of pages to display in the field, then use PAGE_NUMBERS_USING_TOTAL. If you do this, you must specify two numbers somewhere in your TEXT string by the %(flags)(width)d outlined above. In this case, the first number is filled with the current page number, and the second number is filled with the number of total pages.

If you want both the current page number as well as the total number of pages to display in the field, but you want the total number of pages to display before the current page, then use `PAGE_NUMBERS_USING_TOTAL_REVERSE`. If you do this, you must specify two numbers somewhere in your TEXT string by the `%(flags)(width)d` outlined above. In this case, the first number is filled with the number of total pages, and the second number is filled with the current page number.

If you want only the current page number to be displayed in your field and not the total number of pages, then use the `PAGE_NUMBERS` option. In this case, you must specify only one number in your TEXT string by the `%(flags)(width)d` outlined above.

Practical use:

This rule was first used to put the string (continued) on all pages but the last one.

CSCCSetPageNumFieldData

Rule Name

CSCCSetPageNumFieldData

Usage

Generic

Program

csccfld.c

Function Name

CSCCSetPageNumFieldData

Message

RP_POST_PROC_A

Processing Description

For Page Numbering ONLY. This function calculates and prints page numbers by forms. A new page numbering sequence begins for each form.

WHEN options are PRE_PROC or POST_PROC

FLAGS options are same as CSCCSetFieldData - Page Numbering Options only.

Parameter	Usage
Rule Name	CSCCSetPageNumFieldData
Level	2
Add'l Data	FORM->formname\IMAGE->imagename\FIELD->fieldname\TEXT->Page %d of %d\WHEN->POST_PROC\FLAGS->PAGE_NUMBERS_USING_TOTAL\

Example

```
;CSCCSetPageNumFieldData;2;FORM->FORM1\IMAGE->IMAGEFTR\FIELD->PAGE.NO\TEXT
->Page %d of %d\WHEN->POST_PROC\FLAGS->PAGE_NUMBERS_USING_TOTAL\;
```

CSCCSetNonPageNumFieldData

Rule Name

CSCCSetNonPageNumFieldData

Usage

Generic

Program

csccfld.c

Function Name

CSCCSetNonPageNumFieldData

Message

RP_POST_PROC_A

Processing Description

For All Other NON Page Numbering functions. This function performs its requested operation on a form-by-form basis.

This is a rewrite for CSCCSetFieldData, specifically for NON Page Numbers options.

WHEN options are PRE_PROC or POST_PROC
 FLAGS options are ALL_MATCHES or ONLY_FIRST_MATCH or ONLY_LAST_MATCH

Parameter	Usage
Rule Name	CSCCSetNonPageNumFieldData
Level	2
Add'l Data	FORM->FORM1\IMAGE->IMAGE1\FIELD->FIELD-NAME\TEXT->Continued on next Page\WHEN->POST_PROC\FLAGS->ALL_MATCHES\

Example

```
;CSCCSetNonPageNumFieldData;2;FORM->GADECMS PAGE\IMAGE->GASI4\FIELD
->ITEM4.CONTINUED\TEXT->                                \WHEN->POST_PROC\FLAGS->
ONLY_FIRST_MATCH\;
```

CSCCSetPageNumForGroup

Rule Name

CSCCSetPageNumForGroup

Usage

Generic

Program

cscjdt.c

Function Name

CSCCSetPageNumForGroup

Message

RP_PRE_PROC_A
RP_POST_PROC_A

Processing Description

Page Numbering for a list of "FORMS." No Image Level processing in this rule.

This rule is similar to CSCCSetPageNumFieldData except that the page numbering is restricted to the list of FORMS provided in the rule data.

If the "DEC PAGE" is comprised of more than one FORM, then use this rule to compute the page numbering for a GROUP of FORMS. Page Numbering is delimited by FORMBEGIN and FORMEND images.

LIST - comma delimited list of FORM NAMES with an ending comma. This option is optional.

FORMBEGIN - Form to start the page numbering. This option is required. This form MUST be in the formset.

FORMEND - Form to stop the page numbering. This option is required. This form MUST be in the formset

FIELD - Page number field unique to the group of forms in the LIST parameter. It is recommended that the Page Number Field be unique so it is not picked up by any other page numbering rule.

TEXT - Same as CSCCSetPageNumFieldData TEXT options.

WHEN - Same as CSCCSetPageNumFieldData WHEN options.

FLAGS - Same as CSCCSetPageNumFieldData FLAGS options.

Parameter	Usage
Rule Name	CSCCSetPageNumForGroup
Level	2
Add'l Data	LIST->FORM1,FORM2,FORMn,\FORMBEGIN- >FORM1\FORMEND->FORMn\FIELD->CPP- PAGE.NO\TEXT->Page %d of %d\WHEN- >POST_PROC\FLAGS- >PAGE_NUMBERS_USING_TOTAL\

Example

```
;CSCCSetPageNumForGroup;2;LIST->BADECM PAGE,BADECNY PAGE,BADEC2 PAGE,BADEC3
PAGE,BADEC4 PAGE,LOCSCH,SLOCSCH,NASCHED,FORMINV,\FORMBEGIN->BADECM
PAGE\FORMEND->FORMINV\FIELD->CPPPAGE.NO\TEXT->Page %d of %d\WHEN
->POST_PROC\FLAGS->PAGE_NUMBERS_USING_TOTAL\;
```

CUSImportDPXMLFile

CSCCImportDPXMLFile

Rule Name¹

CUSImportDPXMLFile
CSCCImportDPXMLFile

Usage

Generic

Program

cscxml.c

Function Name

CSCCImportDPXMLFile

Level

2

Message

RP_POST_PROC_A

Processing Description

In the Document Print process, when the HTML form is submitted an XML file is created. This rule is executed by GENDATA to import that XML file. If the XML file is not empty and contains matching variable field data for the current document being produced, then the data overrides the data mapped from the extract file.

Preprocessing

Not applicable

Postprocessing

Populate a Global Variable with data from a table read.

Parameter	Usage
Rule Name	CUSImportDPXMLFile or CSCCImportDPXMLFile
Level	2
Add'l Data	Not used

1. This rule is used by Document Print only.

Example

```
;CSCCImportDPXMLFile;2;;
```

CUSSToWIP

CSCCSToWIP

Rule Name

CUSSToWIP

CSCCSToWIP

Usage

Generic

Program

cscjdt.c

Function Name

CSCCSToWIP

Level

2

Message

RP_POST_PROC_A

Processing Description

CSCCSToWIP sends a transaction to WIP based on search criteria.

Parameter	Usage
Rule Name	CUSSToWIP or CSCCSToWIP
Level	2
Add'l Data	Search Criteria

Example

```
;CUSSToWIP;2;TBLOFF,RECORD,FIELD,X;
```

If a record is found that matches the search criteria, the policy is sent to WIP.

CUSTbl2Gvm**CSCCTbl2Gvm*****Rule Name***

CUSTbl2Gvm
CSCCTbl2Gvm

Usage

Generic

Program

cscjdt.c

Function Name

CSCCTbl2Gvm

Level

2

Message

RP_PRE_PROC_A

Processing Description

This rule builds a key from extract file data, reads a table and populates a Global Variable with data from the table read.

Note: Trailing spaces are not trimmed before the data fields for the table key are concatenated together. Table read condition is always "EQ."

Preprocessing

Not applicable

Postprocessing

Populate a Global Variable with data from a table read.

Parameter	Usage
Rule Name	CUSTbl2Gvm or CSCCTbl2Gvm
Level	2

Parameter	Usage
Add'l Data	[TableName],[TableOffset],[Returned Data Length] [Search Criteria] [Offset],[Len][,...],[Global Variable Name]

ADDITIONAL DATA PARAMETERS:

Element Name	Description	Required/Optional
Table Name	Table Name	Required
Table Offset	Identifies the offset into the table data where data retrieval will start.	Required
Returned Data Length	Length of table data to be retrieved.	Required
BLANK	Separator	Required
Search Criteria	Search criteria to find a specific record in the extract file.	Required
BLANK	Separator	Required
Offset	Identifies the offset of a data element to be used to construct the table read key.	Required
Len	Length of the data element to be copied.	Required
[,...]	Can specify more than 1 set of Offset,Len pairs for the table read key build. The data elements are concatenated together.	Optional
Global Variable Name	Global variable that will hold the returned data. The Global Variable must be created with the "CreateGlbVar" rule or be defined in TRNDFDFL.dfd or RCBD-FDFL.dfd.	Required

Error Conditions

Rule sends an error message & returns msgFAIL under the following conditions.

- 1 No parameters exist or the parameters are incorrectly formatted within the Additional Data area.
- 2 Unable to successfully retrieve the Global Variable.

Note: Rule will not return an error if the Search Criteria record is not found.

Example

```
;CSCCTb12Gvm;2;AGTEMAIL,8,50 TBLOFF,PMSP0200 FILLR1,7,EmailAddr;
```

This example does the following:

- Locates the record matching the search criteria "TBLOFF,PMSP0200."
- Obtains the data at offset represented by FILLR1 for a length of 7.
- Builds a key with that data to read into the table called AGTE-MAIL.
- If the table read is successful, 50 characters beginning at offset 8 are returned.
- The returned data is placed in a global variable called Email-Addr.

Note: In this example, the Global Variable called EmailAddr must be defined for a length of 51 (1 character greater than the length of the returned data in order to accommodate the NULL terminating character).

Chapter 4: Job Level 3 Rule Definitions

The DEFLIB member, AFGJOB.jdt, contains the list of job level rules that are executed during GENDAW32.EXE. The four levels of rules are processed in a particular order. The <Sections> are:

<Base Rules>	This base (this implementation) uses these rules.
<Base Form Set Rules>	Every form set in this base uses these rules.
<Base Image Rules>	Every image in this base uses these rules.
<Base Field Rules>	Every field in this base uses these rules.

This chapter is concerned with the rules under the <Base Image Rules> Section header. These rules are executed in the order they appear, processing PRE / POST A messages according to the diagram below.

The placement of the rule under a particular <Section> in AFGJOB and the messages it receives determine what type of processing occurs, not the level indicator in the rule. Each level receives a different set of messages.

You may include comments in AFGJOB.JDT. Comment lines must be on separate lines and in the following format.

```
/* This is a valid comment line          */  
/* ;CreateGlbVar;1;TXTLst,PVOID;         */  
;CSCCConvertImgVar;3;;/* This is an invalid comment line */
```

Processing Diagram

Job Level 3 rules are contained in, but not limited to, rulbsim.c, cusjdt.c, cscjdt.c, pntjdt.c, or clntjdt.c. They receive message A. Base Image Rules, level 3, are called before and after each image is processed.

Base Rules (JL 1) - PRE B	Rulbas/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Formset (JL 2) - PRE A	Rulbsfs/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Image (JL 3) - PRE A	Rulbsim/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Local Image - PRE B	Rulimg/Cusfdt/Cscfdt/Pntfdt/Clntfdt
Base Image (JL 3) - PRE C	Rulbsim/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Field (JL 4) - PRE A	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Local Field - PRE B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - PRE C	Rulbsfl/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Field (JL 4) - POST C	Rulbsfl/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Local Field - POST B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - POST A	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Image (JL 3) - POST C	Rulbsim/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Local Image - POST B	Rulimg/Cusfdt/Cscfdt/Pntfdt/Clntfdt
Base Image (JL 3) - POST A	Rulbsim/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Formset (JL 2) - POST A	Rulbsfs/Cusjdt/Cscjdt/Pntjdt/Clntjdt
Base Rules (JL 1) - POST B	Rulbas/Cusjdt/Cscjdt/Pntjdt/Clntjdt

Job Level Rule Format

Rule Name

Job Level 3 Rule Name Tag

Usage

Generic/Specific

Program

Program

Function Name

Function name

Level

3

Message

RP_PRE_PROC_A
RP_POST_PROC_A

Processing description and pertinent information

Parameter	Max Length	Usage
Rule Name	32	Rule name
Level Number	1	Job rule level number
Add'l Data	1024	Additional data, if necessary

Examples:

;CSCCConvertImgVar;3;POST;

Job Level 3 Rules (Image)

The following table describes all the Job Level 3 rules and where each rule resides. If the function name is different than the rule name, it is noted in the description.

Message

RP_PRE_PROC_A

RP_POST_PROC_A

This section addresses only those rules that were added to CUSLIB or CSCCLIB by CSC. It defines each function and explains how it is used in the context of the system.

Base Rule Name	Function	PGM	Description
CSCCConvertImgVar ConvertImgVar ConvertImageVar	CSCCConvertImgVar	cscjdt	Clear the contents of DDTVarList for each policy transaction.

CSCCConvertImgVar

ConvertImgVar

ConvertImageVar

Rule Name

CSCCConvertImgVar
 ConvertImgVar
 ConvertImageVar

Usage

Generic

Program

cscjdt.c

Function Name

CSCCConvertImgVar

Level

3

Message

RP_PRE_PROC_A
 RP_POST_PROC_A

Processing Description

This Image job rule gives you the capability to use VARs in the rule-parameter area of those image level rules that did not support the use of VARs. Converts VARs during PRE or POST processing.

You can control when the data portion of each image rule is processed and the VARs converted through the INI setting, <PRE_MORPH_IMG_RULE_INCLUSIONS>, in <product>.sys.ini. Only those image rules listed in this INI file group are processed. Designate Pre or Post.

Syntax:

<image-level-rule> = POST | PRE

POST - variable morph occurs on the POST process of the image-level-rule

PRE - variable morph occurs on the PRE process of the image-level-rule.

Variable braces "{}" are converted to "<<>>".

Use UPPER CASE for the rule name.

Example

<PRE_MORPH_IMG_RULE_INCLUSIONS>

RULENAME = PRE

RULENAME = POST

Preprocessing

Convert Image Level VARs for all rules indicated by PRE.

Postprocessing

Convert Image Level VARs for all rules indicated by POST.

Parameter	Usage
Rule Name	CSCCConvertImgVar
Level	3
Add'l Data	Not Used

Example

;CSCCConvertImgVar;3;;

Chapter 5: Job Level 4 Rule Definitions

The deflib member AFGJOB.jdt contains the list of job level rules that are executed during GENDATA.EXE. The four levels of rules are processed in a particular order. The <Sections> are as follows:

<Base Rules>	This base (this implementation) uses these rules.
<Base Form Set Rules>	Every form set in this base uses these rules.
<Base Image Rules>	Every image in this base uses these rules.
<Base Field Rules>	Every field in this base uses these rules.

This chapter is concerned with the rules under the <Base Field Rules> section header. These rules are executed in the order they appear, processing pre/post A and C messages according to the diagram below.

The placement of the rule under a particular <Section> in AFGJOB and the messages it receives determine what type of processing occurs, not the level indicator in the rule. Each level receives a different set of messages.

You can include comments in AFGJOB.JDT. Comment lines must be on separate lines and in the following format.

```
/* This is a valid comment line */  
;CreateGlbVar;1;TXTLst,PVOID; /* This is an invalid comment line */
```

Processing Diagram

Job Level 4 rules are contained in, but not limited to, rulbsfl.c, cusjdt.c, csccejdt.c, pntjdt.c, or clntjdt.c in the order shown below. They can receive message A or C. Only message A is used in the system at this time. Base field rules, level 4, are called immediately before and after local field rules, DDT rules.

Base Rules (JL 1) - PRE B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - PRE A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Image (JL 3) - PRE A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Image - PRE B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - PRE C	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - PRE A	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Field - PRE B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - PRE C	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Field (JL 4) - POST C	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Field - POST B	Rulfld/Cusddt/Cscddt/Pntddt/Clntddt
Base Field (JL 4) - POST A	Rulbsfl/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Image (JL 3) - POST C	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Local Image - POST B	Rulimg/Cusfdt/Csccfdt/Pntfdt/Clntfdt
Base Image (JL 3) - POST A	Rulbsim/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Formset (JL 2) - POST A	Rulbsfs/Cusjdt/Csccejdt/Pntjdt/Clntjdt
Base Rules (JL 1) - POST B	Rulbas/Cusjdt/Csccejdt/Pntjdt/Clntjdt

Job Level Rule Format

Rule Name

Job Level 4 Rule Name Tag

Usage

Generic/Specific

Program

Program

Function Name

Function name

Level

4

Message

RP_PRE_PROC_A
RP_POST_PROC_A

Processing description and pertinent information

Parameter	Max Length	Usage
Rule Name	32	Rule name
Level Number	1	Job rule level number
Add'l Data	1024	Additional data, if necessary

Examples

```
;CreateGlbVar;1;TXTLst,PVOID;  
;Ext2Gvm;2;TBLOFF,PX002001 LOB,3,LOB;  
;RULStandardImageProc;3;Always the first image level rule;  
;RULStandardFieldProc;4;Always the first field level rule;
```

Processing explanation for the rule, other notes, tables, and disclaimers follow.

Job Level 4 Rules (Field)

The following table describes all the Job Level 4 rules and where each rule resides. If the function name is different from the rule name, it is noted in the description.

Message

RP_PRE_PROC_A
RP_PRE_PROC_C
RP_POST_PROC_A
RP_POST_PROC_C

Base Rule Name	Function	PGM	Description
ConvertVar	ConvertVar	cusjdt	Performs morphing on RuleParms data and IF script files prior to the execution of the field level rule

ConvertVar

Rule Name

ConvertVar

Usage

Generic

Program

cscjdt.c

Function Name

ConvertVar

Level

4

Message

RP_PRE_PROC_A

Processing Description

ConvertVar performs morphing on RuleParms data prior to the execution of the field level rule. Some rules are excluded from this process. Refer to FSISYS.ini [PRE_MORPH_EXCLUSIONS] for the current list. Morphing then occurs in the field level rule. Only RuleParm morphing is performed. Morphing for any other DDT line parameter must occur in the rule itself. Morphing is also performed on the initial script files for IF rules.

Acceptable characters to indicate a VAR are “{}” and “<>”. The “{}” are POINT IN specific and “<>” are the CSC Corporate DAP standard. Both are acceptable. Please use “<>” for all future development.

If you want to include a VAR for use with the IF rule, you must enclose the VAR with “<>”. The curly braces have another use in the IF rule. The same restrictions apply to VARs inside an IF script file.

Parameter	Usage
Rule Name	ConvertVAR
Level	4
Add'l Data	Not Used

Example

```
;ConvertVar;4;;
```